University of Lille
IUT of Lille
Informatique département

Development of AI for a Board Game

Internship Report

supported on 06/19/2024

par

Illies Douhab

Kosen supervisor : Koji Tajima

University supervisor : Isabelle Delille

National Institue of Technology(KOSEN), Gifu College

Thanks

I would like to thank everyone who helped to do this internship in Japan

I would thank my internship supervisor, Mr. Tajima, for giving me the opportunity to carry out this internship and this project. Thanks to this, I was able to deepen my knowledge of artificial intelligence and Java sockets.

I would also like to express my gratitude to the Gifu Kosen, its teachers, and its students for their warm welcome and their conducive learning and personal growth environment.

I would like to thank my french internship tutor, Ms. Delille and Ms. Deletombe, for their attentive supervision and valuable assistance throughout this experience. The completion of this project benefited greatly from their invaluable support.

I wish to express my deep gratitude to all my teachers in the BUT Computer Science program for their tireless efforts, which guided my steps throughout this academic year. May all those who supported me, whether closely or from afar, find in this work the expression of my infinite gratitude.

Table des matières

Summary **1**

Introduction **2**

1 Establishment 2
2 Project 2
3 Project management 3
4 Talk Coffee 3
5 The plan 4

1 En Garde ! **5**

1.1 En Garde ! Game 5
1.1.1 Story 5
1.1.2 Rules of the game 6
1.1.3 Client Server Operation 7
1.2 Prerequisites 9
1.2.1 IHM Connexion 9
1.2.2 IHM Play 10
1.2.3 Board 11
1.2.4 Hand 12
1.2.5 Choose a card 13

2 AI **14**

2.1 AI-RANDOM 14
2.2 IA-STATIC 15
2.3 IA-MINIMAX 16
2.3.1 History 16
2.3.2 Fonctionnement 16

2.4 QLearning AI 18
2.4.1 What is the Q-Larning 18
2.4.2 Story and Utilisation 19
2.4.3 Training 19
2.4.4 Porblem 19
2.4.5 Solution 19

3 Ananlyse & Conclusion **20**
3.1 Fight ! 20
3.2 Comparison of results 21

4 Conclusion **22**

Summary

French :

Notre projet consistait en la création d'IA pour le jeu de société En Garde ! en utilisant des algorithmes en Java. Pour réussir ce projet, nous avons divisé en différentes parties. Pour commencer, nous avons d'abord retiré l'IHM du joueur pour simplifier l'implémentation et améliorer les performances des IA. Ensuite, nous avons conçu et implémenté les IA pour le jeu. Le but recherché était la détermination de la meilleure IA pour ce jeu :

— une IA avec un algorithme aléatoire (Random).
— une IA avec un algorithme que j'ai créé (Static).
— une IA basée sur un algorithme existant, le MiniMax (MiniMax).
— une IA basée sur un algorithme existant, le QLearning (QLearning).

Pour finir, nous avons organisé des affrontements entre les IA créées pour savoir laquelle était la plus efficace et la plus intéressante à appliquer.

English :

Our project consisted of creating AIs for the board game En Garde ! using algorithms in Java [1]. To successfully complete this project, we divided it into several parts. First, we removed the player's GUI [2] to simplify implementation and improve the AIs' performance. Next, we designed and implemented the AIs for the game. The goal was to determine the best AI for this game :

— an AI with a random algorithm (Random).
— an AI with an algorithm I created (Static).
— an AI based on an existing algorithm, MiniMax (MiniMax).
— an AI based on an existing algorithm, QLearning (QLearning).

Finally, we organized matches between the created AIs to determine which one was the most effective and interesting to apply.

1. Java is a object-oriented programming language
2. Graphical User Interface

Introduction

1 Establishment

My name is Illies Douhab, and I am currently in my second year of a BUT Informatique, in Application Deployment. My internship is taking place in Japan and started on April 17, continuing until July 10.

During my internship, I had be hosted by the National Institute of Technology (KOSEN) in Gifu (岐阜) city. Established in 1963, this institution now serves about 1,052 students. It is led by Mr. ITSUNO Shinichi. The school consists of five main departments :

— Mechanical Engineering
— Electrical and Computer Engineering
— Electronic Control Engineering
— Civil Engineering
— Architecture

2 Project

We had the opportunity to work with Mr. Tajima, my internship supervisor and computer science professor, on a project. This project consists of developing artificial intelligence for the game "En Garde !". After working on our algorithms, we compared them to determine their effectiveness. This experience allowed us to apply our programming and algorithm create skills.

We all started our project from the same starting point : the game "En Garde !" in its graphical version developed in Java, structured as a client-server application, we have two jar files, as well as their source code. This client-server architecture allowed us to create an interactive gaming environment where each student could develop their artificial intelligence by connecting to their own server.

Our main objective during this internship was to develop artificial intelligence for the game using different algorithms. My work consists of createing, implementing, and evaluating these artificial intelligences to determine which one offered the best performance in the game. Among the challenges encountered were selecting and efficiently implementing the algorithms, as well as optimizing the performance of each artificial intelligence. Additionally, it was necessary to compare and evaluate the performance of each AI to determine which one was the best.

3 Project management

When we arrived, we were integrated into a dedicated Teams group. This group allows us to communicate effectively with our internship supervisor. There we can ask questions, and obtain important information about the project. Additionally, we have the ability to transfer files to it, making it easier to share useful documents and resources. This ensures better collaboration and more efficient monitoring of our progress.

In addition to Teams, during the development of the AI I created a git [3] project on the GitLab [4] of the University of Lille to be able to save my code in the case of a problem, as well as to share it more simply.

4 Talk Coffee

Talk Coffee is an activity that often takes place on Thursday or Friday. During this gathering, students come together to play board games.This activity helps people have fun together and get to know each other better, while creating social connections along the way.

Thanks to these activities, I was able to meet several students, and I discovered new board games, which were unknown to me until now.

3. Git is an open source version control system
4. https://gitlab.univ-lille.fr/illies.douhab.etu/stage

5 The plan

I will begin by explaining what the game "En Garde !" is, including its rules, creator, date of creation, etc ... Next, I will present and explain the Java version of the game, detailing how I removed GUI of the client to simplify the implementation of the AIs. Additionally, I will describe the different AIs and algorithms implemented for the game. Finally, I will compare the efficiency of the AIs using the collected data.

The goal of this project is to be able to answer the question "What is the best AI for En Garde ! game ?"

1

En Garde !

1.1 En Garde ! Game

1.1.1 Story

"En Garde !" is a board game that simulates fencing duels, created by Reiner Knizia and first the game was published in 1993. Reiner Knizia a German board game designer, he is known for his numerous games that often emphasize strategy and mathematical mechanics.

Reiner Knizia created En Garde ! drawing inspiration from classical fencing, seeking to replicate the dynamics and strategy of sword duels in an board game format. The idea was to allow players to experience intense and tactical confrontations using cards to represent movements and attacks.



Figure 1.1 – En Gard ! Board game

1.1.2 Rules of the game

Deploy the dueling track, place the characters at both ends. In front of the dueling track, a shared draw pile of twenty-five shuffled cards (with values 1, 2, 3, 4, 5, in five copies each). Each player draws their initial hand of five cards from it.

In turn, both players must choose an action :

— Move Forward or Backward : The player plays a card and moves their figurine the number of spaces indicated forwards or backwards.

— Direct attack : The player plays one or more cards whose value corresponds to the distance between the figurines. The attacked player must defend by playing the same number and value of cards. If the attacked player cannot defend, they lose the round.

— Indirect attack : The player plays a forward movement card, then one or more attack cards. The attacked player can defend by playing the same number and value of attack cards. If the attacked player cannot or does not want to defend, they must play a backward movement card.

The player with initiative draws cards to replenish their hand to 5 cards. The opposing player, if they have just defended a direct or indirect attack, does not replenish their hand until they have finished their own turn.

If a player cannot defend against a direct attack, they lose the round. If the draw pile is empty, the player furthest engaged on the track wins the round.

The player who win 5 rounds wins the game.

1.1.3 Client Server Operation

The implemented client-server architecture relies on two distinct JAR files : one for the client and one for the server. To start the program, we first launch the server.

Once the server is running, clients connect to it via a socket [5], establishing a network connection between the two. This connection allows the client to send requests to the server and receive responses in a JSon [6] format.

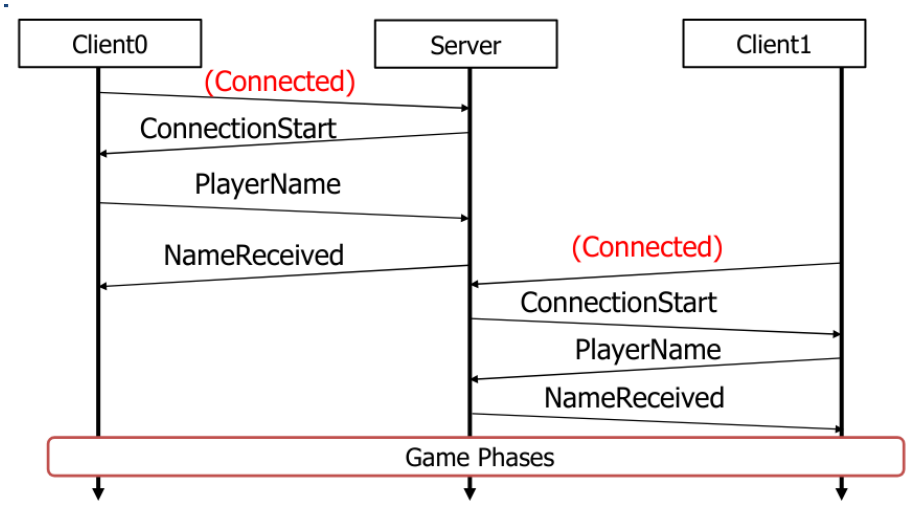


Figure 1.2 – Connection of 2 Client ot the Server

The JSON is used to exchange information between the client and the server. It competes with XML for the representation and transmission of structured information. It will allow communication between the Client and the Server, while remaining readable by a human

Consequently, the client sends the server requests as JSON data upon connection establishment, which are then handled by the server. The client can receive responses as JSON data in return. This communication approach allows an organized and effective flow of information between both ends (client and server), thereby simplifying the system's create and upkeep.

5. A socket is a bidirectional communication interface allowing the exchange of data between two programs
6. JavaScript Object Notation

Example of JSON exchange :

```
[Sent0]{Type=ConnectionStart, ClientID=0, To=Client, From=Server}
[Recv0]{Type=PlayerName, From=Client, To=Server, Name=Client-No-AI}
```

Here is an example of JSON exchange between a client and a server. These two lines are from the server's perspective. So, just after the first client connects with the java socket, the server sends a message to the client to confirm that the connection has been successfully established, and it also sends its ClientID which is 0. The client then responds with its name. In this example, we can see that the client's name is "Client-No-AI".

So, it is under this format that every information will be exchanged between the client and the server, such as : the cards in the player's hand, information about the board, whose turn it is to play, etc ...

1.2 Prerequisites

For the creation of AIs, we need a version of the game without a graphical interface. This will make the create and implementation of the AIs much simpler. To achieve this, we need to understand how the GUI works and identify the actions performed.

1.2.1 IHM Connexion

When we launch the GUI version of the game, it is necessary to specify the ip address [7] and port [8] of the server, which will always be "localhost" [9] and "12052" in our case, because we will launch the server and the two clients from our own machine. To simplify the connection, we will set these parameters so that it happens automatically. The information sent and received by the server will be displayed in the terminal that launched the Java program.

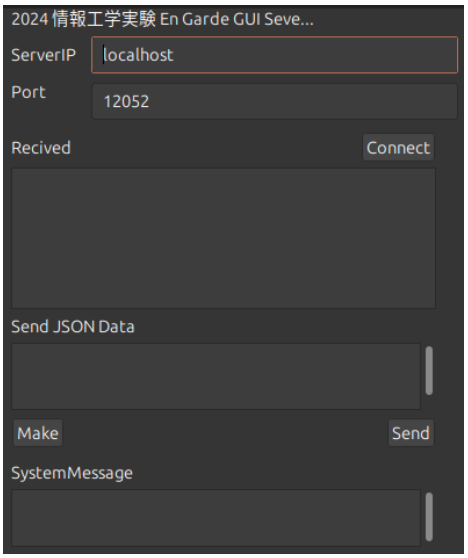


Figure 1.3 – Login page of the GUI version

7. It is a unique value assigned to each device connected to a network
8. Port is a mechanism used in computer networks to uniquely identify applications or services that communicate on a device
9. The IP 127.0.0.1 or "localhost" is an address refers to the local machine itself

Next, when we attempt to connect, a pop-up window opens to ask for the username. To simplify this process, we will predefine the username to match the name of the AI. For example, the random AI will have the name "AI-RANDOM". This approach allows us to differentiate between the various AIs playing without manually specifying the name each time. So, each AI will have a unique and recognizable name, making it easier to track and analyze their respective performances.

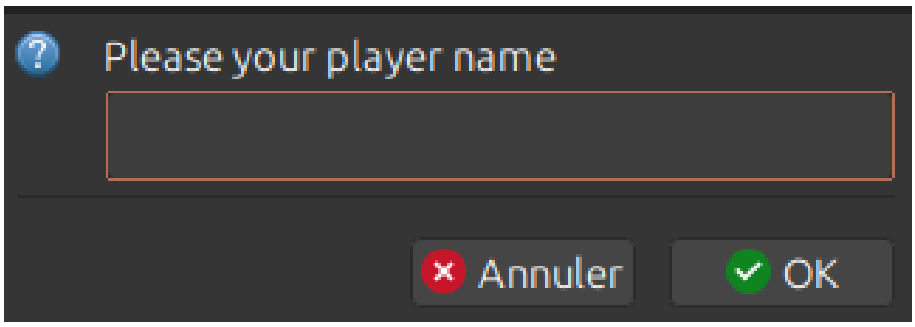


Figure 1.4 – Player name selection

1.2.2 IHM Play

With all the changes made, we can now connect to the server without manual intervention. However, human intervention will be necessary for choosing actions in the game until we integrate AI. For now, the goal is to be able to play without a GUI, using only the terminal.



Figure 1.5 – GUI version of the game

In the image of the game "Figure 1.5," you can see the game's interface. The important information are the board with the two pieces, the cards, and the possible actions. For example, to move forward, you need to select a card (which will be highlighted in red) and press "forward," and to move backward, press "backward." Attacking is a bit different because you can select multiple cards, but they must have the same value. If an action is impossible, an error message appears, allowing you to choose another action. Other information is visible but less important for the game, such as the number of rounds won out of 5 and the number of remaining cards in the deck.

In summary, for the display, we need to show each round, the cards in the player's hand, and the game board. This information will allow the player to know which actions are possible.

Next, the second thing to implement concerns the possible actions : "Forward," "Backward," and "Attack," as well as the card or cards to be used for each action.

1.2.3 Board

For displaying the board, we created a Board.java class that takes the information received from the server each turn.

```
public class Board {
    public String PlayerPosition_1;
    public String PlayerPosition_0;
    public String NumofDeck;
    public String CurrentPlayer;
    public String From;
    public String To;
    public String PlayerScore_0;
    public String PlayerScore_1;
    public char[] board = new char[23];

    ...
```

The information received is in the form of JSON :

```
{"Type":"BoardInfo","PlayerPosition_1":"23","NumofDeck":"15",
"CurrentPlayer":"0","PlayerPosition_0":"1","From":"Server",
"To":"Client","PlayerScore_0":"0","PlayerScore_1":"0"}
```

1.2.4 Hand

Similarly to the board, we created a Hand.java class for managing and displaying the hand, which will update when we receive information from the server.

```
public class Hand {

    public String Hand1;
    public String Hand2;
    public String Hand3;
    public String Hand4;
    public String Hand5;
    public String Type;
    public String From;
    public String To;
    public String[] hand = new String[5];

    ...
```

The information received is in JSON format :

```
{"Hand3":"4","Hand4":"1","Type":"HandInfo","Hand5":"1",
"Hand1":"1","Hand2":"1","From":"Server","To":"Client"}
```

After implementing these two classes, and adding a toString() method which will allow the information to be displayed in the terminal, we obtain this result :



Figure 1.6 – Hand and Board

We can see in "Figure 1.6" the hand with its 5 cards, the board with its 23 spaces, and the position of each player on the board. This information will update each turn so that the player can track the progress of the game.

1.2.5 Choose a card

In each of the two classes, there is a toString() method for displaying the necessary information. Now that we have implemented the display for the hand and the board, we just need to ask the player what action they want to do. To do this, we will use a Java Scanner [10] .

We will ask the following question to determine the desired action : "What do you want to do ? 1 : Forward 2 : Backward 3 : Attack." Then, we will ask : "Choose a card number (1-5)" to select the card to use.

Finally, we have displayed the most important information and we know the move that the player wanted to make, but the server only understands JSON, so we need to put the information that the player wants to send in JSON format. For this, we will use methods from the GUI version of the game which are :

— sendForewardMessage (for move forward with 1 card)
— sendBackwardMessage (for move backward with 1 card)
— sendAttackMessage (for move attack the advairsaire or counter an attack with 1 or several cards)

These three existing methods store the information in a Java Map [11] and then convert it to JSON using the built-in ObjectMapper [12] in Java.

These steps were a little long and time-consuming, but it allowed me to better understand the code and how the java client works, which greatly facilitated the implementation of AI. Thanks to these preparations, the development of AI will now be much simpler and more efficient.

10. A Scanner in Java is a tool that can read user-entered data from the keyboard
11. https://docs.oracle.com/javase/8/docs/api/java/util/Map.html
12. An ObjectMapper in Java is a tool from the Jackson library that allows you to convert Java objects to JSON and also the reverse

2

AI

2.1 AI-RANDOM

To familiarize ourselves with the client code and to start with a simple approach, we began by implementing an artificial intelligence (AI) called "Random-AI". The objective of this AI was to make decisions randomly, without any predefined logic or strategy.

In practice, each action performed by this AI was chosen randomly from the various options available in the game. For example, if the game allowed a player to move or attack, our random AI would simply choose an action at random from these two possibilities, without considering the current state of the game or previous moves.

This approach allowed us to understand the basic mechanisms of the game and the client code, while also giving us our first experience of implementing an AI in this context. By starting with a simple approach, we were able to gradually explore more advanced and complex strategies to improve the performance of our artificial intelligences.

This AI will serve as a baseline for comparison against subsequent AIs, allowing us to evaluate their performance and success rates relative to the Random AI.

The RANDOM AI works but it bases its movements on chance. An AI capable of adapting to the situation thanks to the condition will be much more stronger.

2.2 IA-STATIC

After developing and experimenting with the random AI, we sought to create a more competent artificial intelligence. To do this, we created the STATIC AI, based on an algorithm that I personally invented. This AI is significantly more efficient than the random AI cause of a structured decision path that enables it to take more strategic and thoughtful actions. Here is a detailed explanation of how it operates :

The main objective of the STATIC AI is to improve performance compared to the random AI by integrating a more sophisticated decision logic. Instead of relying solely on randomness, the STATIC AI follows a series of predefined rules that guide its actions in the game.

— Move Forward : The AI selects the card with the highest value among those that appear least frequently in its hand. For example, if the AI has the cards [5, 5, 4, 1, 1], it will choose to play the card 4. This strategy allows the AI to advance significantly while retaining card 5 or 1, which can be more advantageous in fight situations for winning the game.

— Direct Attack : If the distance between the AI and the opponent is less than 5, and the AI has multiple cards of equal value to this distance, then it will play all of these cards. If, for example, AI has the value "2" card 3 times and the distance is 2, she is sure to win because in total there are only 5 value "2" cards. So in the event of a duel using the value card "2" the AI has a 100% chance of winning

— Else : If the AI finds itself in a situation where it cannot go forward or attack. AI will use the card with the smallest value that the AI possesses the least frequently to go backward. For example, if the AI has the cards [5, 5, 2, 1, 1], it will use the card with a value of 2 to go backward.

After organizing confrontations between the STATIC AI and the RANDOM AI, the STATIC AI systematically won all duels. This AI is therefore much more efficient, but it only follows 3 conditions. An AI that can try to simulate a round and know all the possibilities will be much stronger.

2.3 IA-MINIMAX

2.3.1 History

The MiniMax[13] algorithm was first conceived in the 1940s by two eminent figures of game theory, John von Neumann and Oskar Morgenstern. They worked on this algorithmic approach as part of their work when dealing with game theory— a kind of math and economics branch that analyzes strategic interactions among rational agents.

Their work on the MiniMax algorithm became a central piece in the history of artificial intelligence in game playing for its decision-making process in two-player zero-sum games. The algorithm is closely associated with computer chesss, having been studied significantly since its creation and later shown to have applications to other games and decision making problems, becoming a core algorithm in the development to the current state of AI.

2.3.2 Fonctionnement

The MiniMax algorithm is used in two-player games to make the best decisions. Its functioning is grounded on an uncomplicated but efficient rationale :

— Depth-First Search : MiniMax explores the game tree to a certain depth, simulating possible moves for each player from the current game position.

— Position Evaluation : At each level of the tree, the algorithm evaluates the game position based on specific criteria, such as the value of pieces or the probability of winning. It seeks to maximize the score for the current player while minimizing the score for their opponent.

— Backpropagation : Once all possibilities have been evaluated to a certain depth, MiniMax traverses back up the tree to choose the best move to play. It follows the logic of MiniMax : maximizing the score for oneself and minimizing the score for the opponent.

— Best Move Selection : While traversing up the tree, MiniMax chooses the move that maximizes the score for the current player. This means it will choose the move that offers the best possible outcome, even if it involves sacrificing short-term advantages to avoid long-term losses.

13. https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-1-introduction/

In summary, the MiniMax algorithm searches for the best sequence of moves by evaluating the long-term consequences of each decision. It aims to optimize outcomes for the current player while anticipating the opponent's responses. This approach makes it a powerful tool for decision-making in two-player zero-sum games.

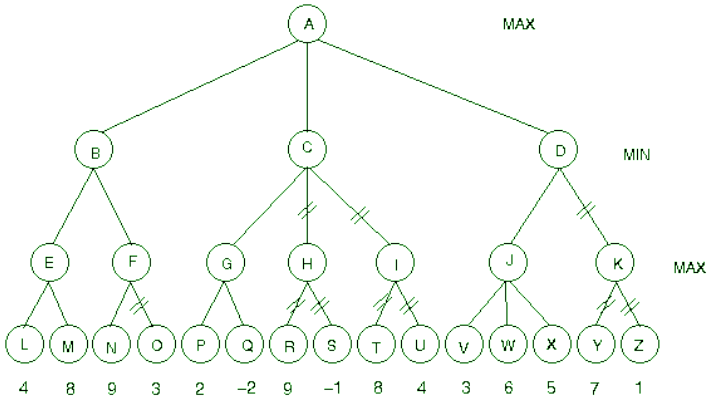


Figure 2.1 – MiniMax algorithm plan

After organizing confrontations between the MINIMAX AI and the STATIC AI, the MINIMAX AI systematically won all duels. This AI is therefore much more efficient, but it only predicts a local fight so if there is an error it will systematically reproduce itself. An AI capable of learning from each fight will be much more stronger.

2.4 QLearning AI

Currently, we have three AIs, each based on a different algorithm. I have explored other types of AI that are not limited to a single algorithm, aiming to create an AI with a different approach. During my research, I discovered AIs that handle games using the QLearning algorithm.

2.4.1 What is the Q-Larning

QLearning[14] is a reinforcement learning algorithm that enables an AI to learn to make optimal decisions by interacting with its environment. This algorithm is based on the concept of Q-values, which represent the long-term utility of performing a specific action in a given state.

The core of the QLearning algorithm is the Q-value update formula, which is given by :

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left(r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right)$$

Here is what each term in the formula represents :

— $Q(s,a)$: The current Q-value of taking action a in state s.
— α : The learning rate ($0 < \alpha \leq 1$), which determines how much new information overrides the old information.
— r : The immediate reward received after taking action a in state s.
— γ : The discount factor ($0 \leq \gamma \leq 1$), which determines the importance of future rewards.
— s' : The new state after taking action a in state s.
— $\max_{a'} Q(s',a')$: The maximum Q-value for all possible actions a' in the new state s'.

14. https://www.freecodecamp.org/news/an-introduction-to-q-learning-reinforcement-learning-14ac0

2.4.2 Story and Utilisation

QLearning was proposed by Christopher Watkins in 1989. Since then, it has been used in various applications, including :

— Video Games : AIs can master complex games using QLearning without needing to know the rules beforehand.
— Robotics : Robots use this algorithm to learn to navigate in changing and unpredictable environments.
— Recommendation Systems : QLearning algorithms help improve online recommendations based on user preferences.

2.4.3 Training

After implementing this new AI, to train it, you need to launch the method and choose the number of rounds the AI will complete for training. The program will then simulate each round of the game locally and record the results in a file named "qtable.ser". To find the best move to play, the program will use this file to determine the optimal move. Therefore, I decided to let it complete 500 training rounds.

2.4.4 Porblem

Two major problems emerge over time with AI. The first is that it is only trained locally against an AI that uses random moves, so it only learns when I activate the training method. The second problem occurs after several hundred training sessions : the program writes and reads the "qtable.ser" file, and the file takes up more and more space with each simulated round.

2.4.5 Solution

After some thinking, I managed to resolve these issues. I improved the AI so that it learns with other AIs, so that it also improves during client-server phases. I changed the handling of the file to limit the effect of the second problem by loading the file once at the start of the game and saving the new information at the end. The program only reads the file once at the beginning and writes it once at the end.

3

Ananlyse & Conclusion

3.1 Fight !

After creating and implementing the four artificial intelligences (AI), I conducted a series of tests to evaluate their performance. These tests involved organizing confrontations between the different AIs, having them play 10 matches consisting of 100 rounds each.

The objective of these confrontations was to compare the results and performances of each AI in a rigorous and statistically significant manner. By observing the results over a large number of rounds, I was able to minimize the impact of variability and random factors, thus obtaining reliable data on the effectiveness of each AI.

To do this, I ran 10 matches of 100 rounds each. At the end of each match, I recorded the results in a PostgreSQL database, which allowed me to calculate the total number of points earned. I then transferred the results to an Excel file for further analysis.

To store the information, I created a PostgreSQL table :

```
Create Table engarde(id SERIAL, name1 Text, score1 Integer,
name2 Text, score1 Integer, totalgame Integer)
```

3.2 Comparison of results

	MiniMax	Static	Random	Qlearning	Wins	Sum of Scores	Rank
MiniMax		636	894	428	2	1958	2
Static	364		844	288	1	1496	3
Random	106	156		75	0	337	4
Qlearning	572	712	925		3	2209	1

Figure 3.1 – Results of AI confrontations

In the Excel file, various information is presented, such as the scores of the AIs during the confrontations, the number of victories, the sum of points earned, and their ranking. These data provide a detailed overview of the performance of each AI, allowing for precise and in-depth comparative analysis.

It is observed that the QLeanrning algorithm is the most effective, with 2209 points earned, closely followed by the STATIC algorithm. However, the RANDOM algorithm is significantly behind the other three.

Après plusieurs milliers d'entraînements, les résultats de l'IA QLearning ont été obtenus. Cependant, l'IA utilisant l'algorithme Minimax était primordiale au départ. Ceci met en évidence deux aspects importants :

— The importance of training for QLearning AI : Reinforcement learning, known as QLearning, requires an analysis of the state and action space to reach an optimal policy. At the beginning of the learning process, the AI lacks sufficient data to make optimal decisions and needs to explore different strategies to deepen its understanding of the consequences of its actions. After participating in many games, the QLearning AI accumulates enough experience to improve its performance and surpass the Minimax AI.

— A well-constructed algorithm : On the other hand, the Minimax AI employs a tree-search algorithm that systematically examines different game positions to determine the most optimal action. It relies on a deterministic and exhaustive approach, making it extremely effective from the start, especially for games with a simple structure like tic-tac-toe or chess. This highlights the importance of having a solid and well-created foundational algorithm for any AI.

4

Conclusion

To conclude this report and address the central question : what is the best AI for the game En Garde ! ?

To recap, we created and implemented four AIs, each functioning differently :

— Random AI : Selects its actions randomly, offering no strategic depth and serving as a baseline for comparison.
— Logic-based AI : Operates on a basic set of predefined rules, providing a more structured method than the Random AI but lacking advanced decision-making capabilities.
— MiniMax AI : Utilizes the MiniMax algorithm, which generates a decision tree with all possible actions and selects the best move based on a heuristic evaluation function. This AI employs a sophisticated approach, considering future game states to make optimal decisions.
— QLearning AI : Uses a reinforcement learning algorithm that learns from experience. This AI improves its performance over time by training and adjusting its strategy based on rewards received from different actions.

We developed and implemented these four algorithms for the game and had them compete against each other. The results of these confrontations were recorded in an Excel file for comparison.

Our findings indicate that each AI has its strengths and weaknesses :

— Random AI : As expected, this AI performed the weakest, as it lacked any strategic planning.
— Logic-based AI : This AI performed better than the Random AI but was still limited by its simplistic ruleset.
— MiniMax AI : This AI showed strong performance due to its ability to consider multiple future moves and choose the optimal action. However, it requires significant computational resources, especially as the depth of the decision tree increases.
— QLearning AI : Initially, this AI performed poorly due to its need for extensive training. Over time, however, its performance improved significantly, eventually surpassing the Logic-based AI and approaching the effectiveness of the MiniMax AI.

In summary, the evolution of the QLearning AI's performance demonstrates the importance of reinforcement learning, highlighting its ability to adapt and improve over time. However, the MiniMax AI's consistent strength underscores the importance of an effective initial algorithm for achieving competitive results from the start. Therefore, the best approach to developing robust and high-performing AIs for En Garde ! involves combining a sophisticated base algorithm like MiniMax with an intensive training phase for reinforcement learning. This hybrid approach leverages the immediate strength of MiniMax and the adaptive improvement of QLearning, resulting in a highly capable AI.